

Cohort module four, two hours

Make Agents Prove Their Work

The longest build of the course. Everyone finishes with checks that fail closed and a reviewer that reads their diffs without being told the story.

0:00 to 0:30

One script, one result line

Goal Everyone's checks are behind one command that says one thing, and a missing check is louder than a failing one.

Say

- Start from where they are. Most have commands scattered across a pipeline file, a readme and somebody's shell history.
- Gather them into one script live, then add the ending that makes it useful: a single line, exactly worded, that nothing may restate.
- Do the deletion demonstration on their own project, not the lab. Remove a required command and show that the result is neither green nor a failure.

Run

```
$ bash steps/04-verification/gates.sh
$ bash steps/04-verification/gates.sh --without-tests
$ echo $?
```

Room does

- Writes one script for their own repository with the same ending.
- Confirms three distinct exit codes for passing, failing and misconfigured.

Ask *Which of your checks is currently optional, and who decided that?*

0:30 to 1:00

Tests the writer never sees

Goal Each person has a small set of behavioural checks kept out of the implementing agent's reach.

- Say**
- Explain the circularity plainly: when the same run writes the tests and the code, green means the code agrees with itself.
 - Choose what to hold out together. The contract and the behaviour a user would notice, not the internals.
 - Head off the objection before it arrives. This is a sample, not a second suite, and a handful is enough.
-

Run

```
$ ls steps/04-verification/holdout
$ node steps/04-verification/demo.ts holdout
```

- Room does**
- Sees the holdout set run after the change, against work that never saw it.
 - Picks three behaviours in their own project worth holding out.
-

Ask *What behaviour would embarrass you most if it broke quietly? Start there.*

1:00 to 1:30

The check no gate can make

Goal Everyone has run it against a test they already trusted, and some of them are surprised.

- Say**
- Have them pick a test they wrote themselves and are confident in. The confidence is the point.
 - Walk the sequence together: undo the change it covers, run it, expect red, restore.
 - Ask who found one that stayed green. There is usually at least one, and that discovery does more teaching than the explanation.
-

Run

```
$ node steps/04-verification/demo.ts negative-proof
$ node steps/04-verification/demo.ts negative-proof --toothless
```

- Room does**
- Applies the sequence to one of their own tests by hand.
 - Wires it into their loop so it happens on every change rather than when remembered.
-

Ask *How many of your tests have never once been red? What does that number mean?*

1:30 to 2:00

Deciding what a result may claim

Goal	Their pipeline downgrades a conclusion whose evidence is missing, without consulting what the agent said.
Say	• Separate the two jobs sharply. Investigation is where judgement helps. The sentence a person will act on gets no freedom at all. • Write the rules as ordinary code in front of them, reading the artifacts the run left behind. • Connect it to the cross-language case: a definition changed in one place, and the proof required is three suites agreeing, not one claim that it went fine.
Run	<pre>\$ node steps/04-verification/demo.ts evidence \$ node steps/04-verification/demo.ts evidence --lost-artifact</pre>
Room does	• Sees a confident conclusion downgraded because its proof could not be found. • Writes one such rule for their own repository.
Ask	<i>What does your pipeline currently believe purely because something reported it?</i>

1:35 to 1:50

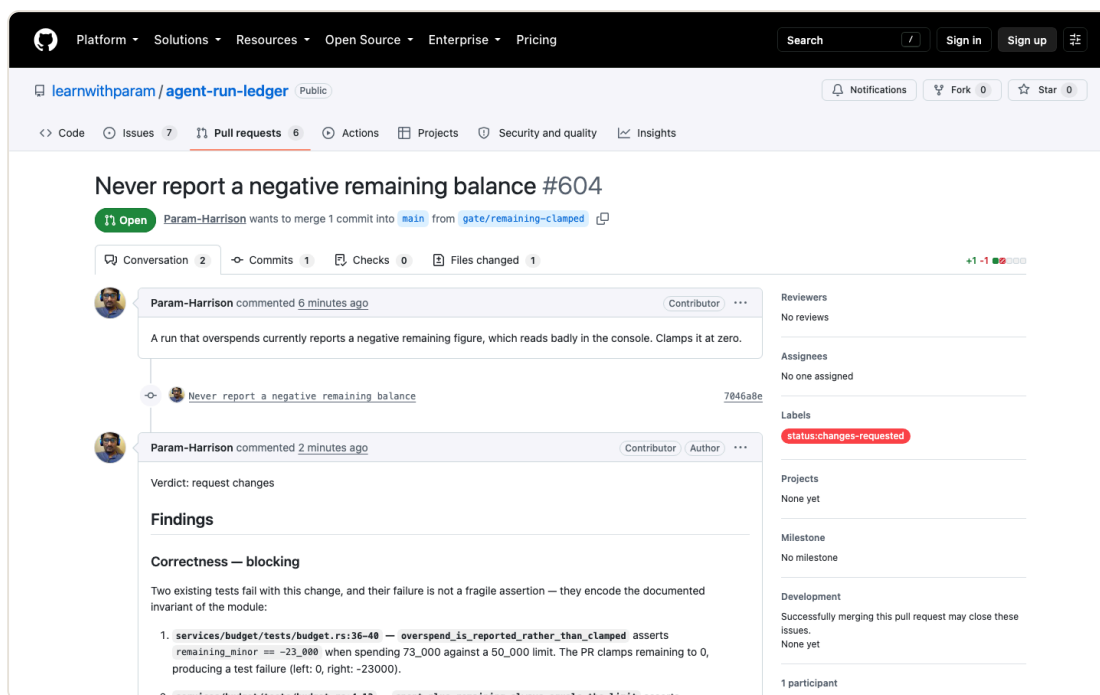
A gate failing in one language

Goal They can defend running only the checks a change reaches, and name what that misses.

- Say**
- The ownership graph decides which suites a change is measured by, so a schema edit is judged by all three.
 - Show the failure and note that the language it failed in is not the language the issue named.
 - Then name the cost honestly: a wrong graph means a real failure nobody ran.

Run

```
$ make check
```



One change, three suites, and only one of them red.

Show on the board

Work to the cross-stack item to its session to three suites, one change

Only the suites a change reaches

The ownership graph decides what runs. A schema edit is judged by all three languages; a console change is judged by one. Show which ran and which did not.

A wrong graph is a failure nobody sees

If a directory is claimed by the wrong target, its suite never runs against changes that reach it. That is the cost of this design and it belongs in the room rather than in a footnote.

The verdict is quoted, never summarised

The record carries the gate's own last line. A summary is a second source of truth about whether something passed.

Room does

- Reads which suites ran and which did not.
 - Names one path in their repository whose owner is currently ambiguous.
-

Ask

If your graph were wrong by one directory, which failure would you never see?

Concepts: the result line, held-out checks, an independent read, proving the test bites, evidence rules, one definition.